

```

        adapter.record(message);
    }

    @Override
    public String getType() {
        return "com.glarimy.ums";
    }
}

```

The corresponding implementation of *UserRepositoryJournalProxy* is:

```

package com.glarimy.ums;

import com.glarimy.framework.Broker;
import com.glarimy.framework.Factory;
import com.glarimy.framework.ObjectFactory;

public class UserRepositoryJournalProxy implements
UserRepository {
    private UserRepository target;
    private Broker broker;
    private static final String TOPIC = "com.glarimy.ums";

    public UserRepositoryJournalProxy(UserRepository target)
throws Exception {
        this.target = target;
        Factory factory = new ObjectFactory("config.
properties");
        this.broker = (Broker) factory.get("broker");
        this.broker.register(new JournalHandler());
    }
    ...
}

```

Being a proxy, the *UserRepositoryJournalProxy* publishes messages to the broker as part of its pre- and post-processing routines.

```

@Override
public void add(Long phone, String name) {
    broker.publish("c=repository, s=add, a=" + "&" + name,
TOPIC);
    target.add(phone, name);
    broker.publish("c=repository, s=add, r=void", TOPIC);
}

@Override
public String find(Long phone) {
    broker.publish("c=repository, s=find, a=" + phone,
TOPIC);
    String name = target.find(phone);
}

```

```

broker.publish("c=repository, s=find, r=" + name, TOPIC);
return name;
}

```

The updated design of the UMS is depicted in Figure 3.

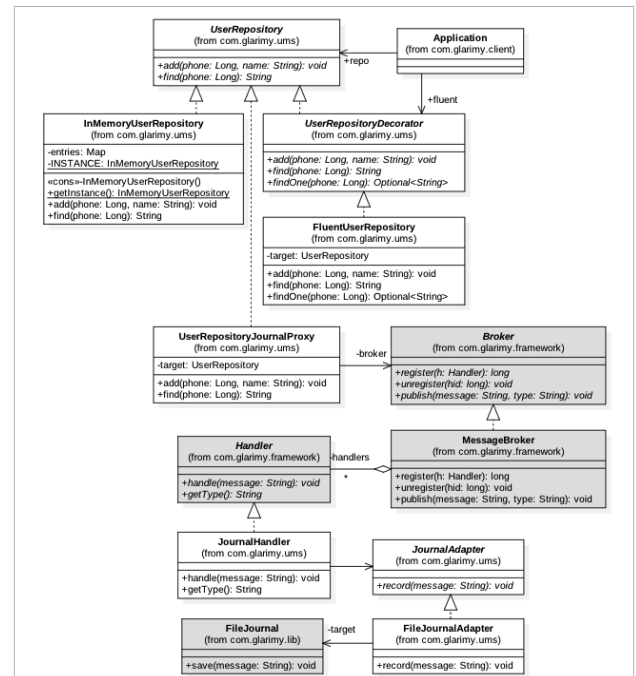


Figure 3: UMS with messaging

An important observation is that the *UserRepositoryJournalProxy* and the *JournalAdapter* are now completely decoupled. In fact, in response to the messages published by the *UserRepositoryJournalProxy*, any number of handlers on the topic *com.glarimy.ums* can respond by their own processing. For example, a handler may send an email and another handler may send an SMS confirming the addition of a user. Registering or unregistering such handlers does not have any bearing on the rest of the system.

If the broker invokes the handlers in separate threads, this implementation realises a complete non-blocking asynchronous communication.

With this, we leave the object-oriented design using GoF patterns. In the next part, we will explore domain-driven design. **END** 🐧

By: Krishna Mohan Koyya

The author is the founder and the principal technology consultant at Glarimy Technology Services, Bengaluru. He has delivered more than 500 programs to corporate clients on design and architecture in the last 12 years. Prior to that, he held various positions in the industry and academia for a decade.